ChatGPT

Riconoscimento dello Stile Architettonico con CLIP e PyTorch

Introduzione al Problema

Il riconoscimento dello **stile architettonico** da fotografie è un compito di classificazione di immagini in cui ogni edificio va assegnato a una categoria stilistica (es. Gotico, Barocco, Moderno, ecc.). Si tratta tipicamente di un problema **multi-classe** (un edificio appartiene principalmente a uno stile), ma in alcuni casi può essere formulato come **multi-etichetta** (uno stesso edificio potrebbe mostrare caratteristiche di più stili). Le sfide includono la variabilità nelle inquadrature (diverse prospettive e angolazioni), condizioni di illuminazione, decorazioni similari tra stili differenti e la disponibilità limitata di immagini etichettate per alcuni stili meno comuni.

Per affrontare questo progetto si è scelto di utilizzare **CLIP** (Contrastive Language-Image Pre-Training) come modello backbone. CLIP è un modello multimodale addestrato su 400 milioni di coppie immagine-testo, capace di mappare immagini e descrizioni testuali in uno spazio di embedding comune [1] [2]. Questa caratteristica consente a CLIP di svolgere **classificazione zero-shot**, ovvero riconoscere classi mai viste esplicitamente durante l'addestramento, semplicemente scegliendo la descrizione testuale più simile all'immagine fornita [3]. Nel contesto degli stili architettonici, CLIP può ad esempio confrontare una foto di un edificio con descrizioni del tipo *"una foto di un edificio in stile Gotico"* vs *"...in stile Moderno"* ecc., e identificare quella con embedding più affine all'immagine.

Di seguito verrà delineata una pipeline completa che comprende: - **Preprocessing** delle immagini (es. filtraggio del rumore, correzione prospettica, rilevamento e ritaglio dell'edificio nella foto). - **Estrazione di embedding** con CLIP. - Una possibile fase di **image retrieval** per confrontare l'immagine con un database di esempi noti. - La **classificazione finale** dello stile architettonico, sia in modalità multi-classe che (eventualmente) multi-label.

Verranno inoltre discussi: 1. Documentazione e indicazioni sull'uso di CLIP per estrarre feature e classificare immagini in questo contesto. 2. Esempi di implementazioni simili (open-source) con PyTorch. 3. Un'analisi dei pro e contro tra l'uso di CLIP *pre-addestrato* in zero-shot vs. l'approccio di *fine-tuning* supervisionato. 4. Dataset disponibili (sia quelli citati che alternativi/complementari). 5. Best practice per costruire la pipeline e metriche per valutare i risultati.

Utilizzo di CLIP per Feature Extraction e Classificazione

CLIP fornisce due modalità principali d'uso nel nostro scenario: come **estrattore di feature** visive e come **modello di classificazione zero-shot**.

- **Estrazione di embedding con CLIP:** Possiamo utilizzare l'encoder di immagini di CLIP per convertire ogni foto in un vettore di caratteristiche (embedding) nello spazio latente multimodale. In PyTorch, ad esempio, tramite la libreria HuggingFace è possibile caricare il modello CLIP pre-addestrato (*vision encoder* ViT-B/32, ViT-L/14, ecc.) e processare un'immagine ottenendo l'embedding come tensore. Il codice seguente illustra il processo: si carica il modello CLIP e il relativo processor, si fornisce in input un'immagine e alcune descrizioni testuali

candidate, e si ottengono similitudini immagine-testo [4] [5]. In output, `logits_per_image` contiene lo **score di similarità** tra l'immagine e ciascun testo; applicando softmax si ottengono le probabilità per ogni classe testuale candidata. Questo approccio consente di realizzare facilmente la **classificazione zero-shot**, come descritto anche nella documentazione di HuggingFace e vari tutorial [6] [7].

- **Classificazione zero-shot con prompt testuali:** CLIP eccelle nel confrontare immagini e testi, permettendo di classificare un'immagine scegliendo tra varie descrizioni. In pratica, per ciascuno stile architettonico si formula un *prompt* (es. *"una foto di un edificio in stile Gotico"*, *"...in stile Barocco"*, ecc.) e si calcola la similarità tra l'immagine e ogni descrizione. L'etichetta con embedding più simile all'immagine viene predetta come output [2]. Questo metodo **non richiede alcun addestramento supervisionato** sul dataset specifico (da cui *zero-shot*). CLIP, addestrato su moltissimi dati, possiede già conoscenze visivo-semantiche ampie: è infatti in grado di classificare direttamente immagini in una varietà sorprendente di categorie, anche mai viste durante il training, purché le categorie siano descritte in linguaggio naturale [3]. Nel nostro caso, se lo stile è un concetto presente nei dati di addestramento di CLIP (ad esempio *"Gothic architecture"* è un termine abbastanza noto in inglese), il modello può coglierne le caratteristiche visive senza ulteriore training.

- **Uso di CLIP come backbone feature extractor:** In alternativa allo zero-shot puro, si può utilizzare CLIP per **estrarre feature** e poi allenare un classificatore tradizionale sopra di esse. In questa modalità, CLIP funge da estrattore di rappresentazioni robuste delle immagini; successivamente si aggiunge un layer (ad es. una rete feed-forward o anche un semplice classificatore lineare tipo SVM o Logistic Regression) che, dato l'embedding di CLIP, predice lo stile architettonico. Questo approccio richiede un dataset etichettato per allenare il classificatore finale, ma **mantiene congelati i pesi di CLIP** (facendo quello che si chiama *fine-tuning* limitato o *linear probing*). Il vantaggio è che la rete CLIP pre-addestrata fornisce già ottime astrazioni visive, quindi spesso basta allenare per poche epoche un livello finale per ottenere buone performance [8]. Ad esempio, un recente confronto ha mostrato che con poche epoche di fine-tuning CLIP può superare modelli tradizionali (come ResNet50) allenati più a lungo sullo stesso task [9]. In altri termini, CLIP porta una conoscenza visuale molto generale che aiuta a classificare bene anche dataset specifici dopo minima ottimizzazione dei pesi [9].

- **Fase di retrieval (facoltativa):** Un ulteriore impiego di CLIP è l'**image retrieval** ossia il recupero di immagini visivamente simili mediante confronto tra embedding. Dopo aver estratto l'embedding CLIP di una foto di input, si può cercare le immagini più vicine in un database di edifici catalogati per stile (usando ad esempio la distanza coseno). Questa tecnica può servire a due scopi: (1) effettuare classificazione mediante *k-nearest neighbors* (assegnando lo stile più frequente tra i K vicini più simili), oppure (2) fornire spiegazioni/contestualizzazioni mostrando esempi simili dal database. CLIP è stato progettato proprio per favorire queste applicazioni di **visual search**, grazie al suo spazio congiunto immagine-testo [10]. Implementativamente, si potrebbe indicizzare gli embedding di immagini di riferimento (magari alcune decine per ciascuno stile noto) in uno spazio vettoriale (usando strumenti come FAISS per ricerca veloce). Alla richiesta di una nuova immagine, CLIP genera l'embedding e si recuperano i più vicini nell'indice; se ad esempio la maggior parte dei vicini appartiene allo stile *Gotico*, si può concludere che anche l'immagine query è gotica. Questa fase di retrieval può aumentare la robustezza della pipeline, fungendo da secondo parere o integrando la classificazione (ad es. votazione tra classificatore e nearest-neighbors). Dal punto di vista computazionale, è un passaggio opzionale ma utile soprattutto quando si hanno **molte classi** o se si vuole supportare una modalità di esplorazione interattiva delle immagini simili.

Esempi di Implementazioni Open-Source in PyTorch

Per comprendere meglio come strutturare il progetto, è utile esaminare lavori esistenti simili, sia in ambito accademico che community. Di seguito alcuni esempi e risorse utili:

- **Architectural Style Recognition (FastAI/PyTorch):** un progetto open-source di classificazione di 25 stili architettonici realizzato con la libreria FastAI (che si basa su PyTorch) [11]. Questo progetto utilizza un modello pre-addestrato (ResNet-101) riaddestrato sul dataset di 25 classi (vedi sezione dataset) tramite transfer learning [12]. L'autore riporta i dettagli del dataset utilizzato (10.113 immagini di 25 stili, combinando immagini web e un dataset pubblicato in letteratura) [13] e descrive le tecniche usate per migliorare le prestazioni, come l'**data augmentation** (flip orizzontale, zoom, variazioni di luminosità, rotazioni, ecc.) [14]. Grazie a queste tecniche e al fine-tuning del modello, questa implementazione dichiara di aver raggiunto un'accuratezza del **95%** su tale dataset [15] – un risultato molto elevato rispetto al ~70% ottenuto dai metodi classici sullo stesso dataset nel 2014 [16]. Il codice e la documentazione di questo progetto (disponibile su GitHub) possono fornire spunti sia per la preparazione dei dati che per l'architettura della rete finale (in quel caso un semplice classifier feed-forward sul backbone ResNet).

- **Roboflow ArchiStyle e ArchiTective:** sulla piattaforma Roboflow Universe sono disponibili sia dataset che modelli pre-addestrati per il riconoscimento di stili architettonici. In particolare, il dataset **ArchiStyles** (curato dall'utente *mipt4*) contiene migliaia di immagini annotate in 25 categorie di stile, e un modello pre-addestrato raggiunge circa il **73,6% di accuratezza** su queste classi [17]. Un altro progetto denominato **ArchiTective** si concentra su 4 stili (Art Nouveau, Gotico, Decostruttivismo e Palladiano) ed offre un modello addestrato su ~1800 immagini di queste quattro classi [18] [19]. Questi modelli Roboflow possono essere sfruttati come baseline di confronto; inoltre il formato dei dataset su Roboflow (con suddivisione train/val, augmentations applicabili via API, ecc.) può facilitare la fase di raccolta e preprocessing delle immagini.

- **Esempi di utilizzo di CLIP (OpenAI/HuggingFace):** Il repository ufficiale OpenAI CLIP e implementazioni open-source come **OpenCLIP** (di LAION) forniscono codice per impiegare CLIP in vari task. Ad esempio, il repository *openai/CLIP* su GitHub contiene notebook dimostrativi per la classificazione zero-shot su ImageNet tramite CLIP. Allo stesso modo, HuggingFace Transformers mette a disposizione pipeline pronte all'uso: con `pipeline(task="zero-shot-image-classification", model="openai/clip-vit-base-patch32")` è possibile classificare immagini passando direttamente le possibili label testuali, come mostrato in questo tutorial [6] [20]. Queste risorse documentano come preprocessare le immagini (ridimensionamento e normalizzazione come richiesto dal modello CLIP) e come costruire i prompt testuali ottimali. Si trovano anche esempi di **fine-tuning di CLIP**: ad esempio, HuggingFace e blog come *Marqo* spiegano come continuare l'addestramento di CLIP su un dataset personalizzato [21]. Un progetto su GitHub dedicato ("CLIP Fine-Tuning with PyTorch" di Muhammad Ali) fornisce script per allenare CLIP su task di classificazione immagini e persino Visual Question Answering [22]. Questi esempi open-source possono servire come riferimento per impostare la fase di training nel nostro progetto, in particolare se si decide di fine-tunare CLIP invece di usarlo solo come feature extractor.

- **Altri progetti correlati:** In letteratura recente, *CLIP* è stato applicato anche in ambiti affini. Ad esempio, uno studio introduce **ArchiCLIP**, dove CLIP viene **fine-tunato con dati di architettura (oltre 1 milione di coppie immagine-testo di progetti architettonici)** per specializzarlo nel riconoscimento di caratteristiche architettoniche [23]. I risultati mostrano che un CLIP adattato al dominio architetturale **supera il CLIP pre-addestrato** in compiti come la classificazione di stile

[24]. Questo conferma la bontà dell'approccio di fine-tuning su dati specifici: se si dispone di un dataset sufficientemente ampio e vario, *personalizzare* CLIP con conoscenza di dominio può portare benefici significativi (ArchiCLIP è usato anche per migliorare modelli generativi di immagini architettoniche, a testimonianza della qualità delle embedding specializzate ottenute [25]). Pur non essendo un progetto direttamente utilizzabile (il dataset custom non è pubblico al 100%), ArchiCLIP indica una direzione promettente, e la descrizione fornita nel relativo paper può offrire indicazioni su come curare un dataset testuale+visivo per l'addestramento contrastivo.

Fine-Tuning di CLIP vs Classificazione Zero-Shot: Pro e Contro

Una decisione chiave è se utilizzare CLIP **così com'è (zero-shot)** oppure **fine-tunarlo** sul nostro dataset di stili architettonici. Entrambi gli approcci hanno vantaggi e svantaggi, e la scelta può dipendere dalle performance ottenute sperimentalmente e dalle risorse disponibili. Di seguito un'analisi comparativa:

- **Zero-Shot (CLIP pre-addestrato, senza ulteriori training supervisato):** *Vantaggi*: Non richiede dati etichettati né tempo di addestramento sul task specifico – il modello è pronto all'uso. È estremamente flessibile: possiamo aggiungere o modificare le classi semplicemente cambiando o aggiungendo descrizioni testuali, senza riaddestrare nulla. Inoltre, sfrutta al massimo la conoscenza "enciclopedica" appresa da CLIP su immagini internet: CLIP è in grado di riconoscere una vasta gamma di concetti visivi direttamente tramite descrizioni in linguaggio naturale [3]. Nel caso di stili ben noti (es. *Gothic architecture* o *Baroque church*), il modello potrebbe già saperli distinguere in base a ciò che ha appreso durante il pre-training. *Svantaggi*: La performance potrebbe essere inferiore rispetto a un modello specializzato sul nostro dataset. In particolare, **classi fine-grained** o molto simili tra loro possono confondere CLIP in zero-shot. Gli autori originali di CLIP notarono che su compiti di classificazione molto fini (ad es. distinguere modelli di aerei da fotografie) il modello base rende poco [26]. Gli stili architettonici presentano talvolta differenze sottili (si pensi a *Georgiano* vs *Coloniale*, o altri stili di epoche contigue) e CLIP potrebbe non avere abbastanza "grip" su queste differenze senza un adattamento. Inoltre, i risultati zero-shot dipendono molto dai *prompt* testuali scelti: trovare la formulazione ideale ("foto di edificio in stile X", "architettura X", ecc.) incide sui punteggi di similarità. È possibile migliorare leggermente le prestazioni zero-shot con **prompt engineering** (variare descrizioni, aggiungere contesto ai testi, ecc.), ma resta un limite il fatto che il modello non è stato ottimizzato specificamente sulle nostre categorie. In sintesi, l'approccio zero-shot offre **immediatezza e flessibilità**, ma potrebbe sacrificare accuratezza, specialmente se i nostri dati differiscono dallo "stile" di dati visti da CLIP (ad esempio, se abbiamo molte foto amatoriali o dettagli architettonici non presenti nei dati di pre-training).

- **Fine-Tuning di CLIP sul dataset di stili:** *Vantaggi*: Permette di massimizzare le prestazioni sul nostro compito specifico. Adattando i pesi del modello alle nostre classi, CLIP può imparare differenze sottili tra stili che in modalità zero-shot non distingueva bene. Studi empirici mostrano che CLIP fine-tunato può superare la versione zero-shot di parecchi punti percentuali in accuracy [27], colmando il gap su classi niche o fine-grained. Inoltre, CLIP tende a convergere in poche epoche grazie alla sua buona inizializzazione [8]: anche con dataset non enormi, pochi passi di gradiente possono apportare miglioramenti significativi, sfruttando la sua "broad understanding" iniziale [9]. Nel nostro caso, potremmo aspettarci che un fine-tuning su, ad esempio, 5-10 epoche del dataset ArchiStyles migliori sensibilmente l'accuratezza rispetto al zero-shot (potenzialmente avvicinandosi o superando i ~95% riportati con ResNet in progetti precedenti [15]). *Svantaggi*: La necessità di disporre di un dataset di addestramento etichettato e il costo computazionale dell'addestramento. CLIP non è leggerissimo: la versione base ViT-B/32 ha ~90 milioni di parametri (circa 4 volte una ResNet50 [28]) e la versione ViT-L/14 supera i 300M.

Fine-tunare tali modelli richiede GPU con buona memoria e tempo di calcolo. Inoltre c'è il rischio di **overfitting** se il dataset è ridotto: CLIP può adattarsi troppo alle immagini di training e perdere capacità di generalizzazione (soprattutto se si eseguono molte epoche senza tecniche di regularization). Bisogna quindi impostare il fine-tuning con attenzione: tipicamente si usa un learning rate basso, eventualmente congelando alcune layer iniziali dell'encoder e allenando prima solo gli ultimi strati, per poi sbloccarli gradualmente. Un altro potenziale svantaggio è la **specializzazione eccessiva**: un CLIP fine-tunato potrebbe *dimenticare* parte della sua conoscenza generale (fenomeno di *catastrophic forgetting*). In pratica, se poi volessimo riutilizzarlo per riconoscere concetti al di fuori degli stili architettonici, le performance zero-shot potrebbero degradare. Questo problema però si mitiga se il fine-tuning è moderato o se manteniamo una copia del modello originale per altri scopi. In definitiva, il fine-tuning comporta un investimento di tempo e risorse, ma **massimizza l'accuratezza** e adatta il modello al nostro dominio.

- **Approccio intermedio – CLIP come feature extractor + classificatore:** Vale la pena menzionare una terza via, a metà tra zero-shot e fine-tuning completo: usare CLIP congelato come estrattore di feature e allenare solo un classificatore finale (ad es. un layer lineare) sulle nostre classi. Questo approccio richiede comunque dati etichettati, ma molto meno tempo computazionale rispetto a riaddestrare tutto il modello. In sostanza, si evitano i rischi di overfitting delle feature di basso livello e si allena solo la "testa" di rete per tararla sulle classi del dataset. Il pro è che spesso già così si ottengono performance elevate (CLIP ha embedding ricchi); il contro è che potrebbe non raggiungere il massimo possibile, specie se le caratteristiche rilevanti per distinguere alcuni stili non sono già perfettamente codificate nell'embedding originario. Comunque, allenare un classificatore lineare su CLIP embedding può essere un buon **punto di partenza**: se i risultati sono soddisfacenti (diciamo, l'accuracy raggiunge un livello accettabile), potremmo evitare il fine-tuning completo. Viceversa, se vediamo che manca ancora accuratezza, allora passare al fine-tuning end-to-end diventa giustificato.

In conclusione, si può iniziare valutando CLIP in zero-shot: se già ottiene performance vicine al nostro obiettivo, potremmo utilizzare quello (magari raffinando i prompt testuali). In caso contrario, l'approccio raccomandato è fare fine-tuning (almeno del classificatore finale) sul dataset di interesse e confrontare. **In molti casi pratici, la combinazione di CLIP + fine-tuning leggero offre il migliore trade-off** tra sforzo e accuratezza: ad esempio, CLIP dopo poche epoche di training su dati di dominio ha eguagliato/superato modelli classici addestrati da zero, dimostrando il valore del fine-tuning nel captare sfumature specifiche [8].

Per chiarezza, ecco una tabella riassuntiva dei principali pro/contro:

Approccio	Pro	Contro ⚠
CLIP Zero-Shot	- Nessun bisogno di training nè dati etichettati. - Flessibile: basta cambiare/aggiungere descrizioni per nuove classi. - Sfrutta conoscenza generica ampia di CLIP [3].	- Accuratezza potenzialmente inferiore su classi fine-grained [26]. - Sensibile alla formulazione dei prompt testuali. - Non ottimizzato per il dataset specifico (può confondere stili simili).

Approccio	Pro	Contro ⚠
CLIP Fine-Tunato	- Massimizza le prestazioni sul compito specifico (migliore discriminazione tra stili simili) [27] . - Poche epoche di training possono bastare grazie alla buona inizializzazione [9] . - Adattabile: si può effettuare fine-tuning parziale (es. solo ultimo layer).	- Richiede dataset etichettato e risorse computazionali per l'addestramento. - Modello pesante (CLIP ViT-B/32 ~4× ResNet50 in dimensioni) [28] . - Rischio overfitting se il dataset è piccolo; richiede tuning accurato di iperparametri.

Dataset Disponibili per l'Addestramento

Una componente fondamentale è la disponibilità di **dataset** di immagini di edifici etichettate per stile architettonico. I dataset citati nella domanda e altri complementari sono:

- **ArchiStyles (Roboflow/Kaggle)** – Un dataset ampiamente utilizzato in letteratura e community, originato dal lavoro di Z. Xu et al. (ECCV 2014) e successivamente esteso. Contiene ~**10.113 immagini** appartenenti a **25 stili architettonici** differenti [13] (dall'Antico Egitto a stili moderni come Bauhaus e Postmoderno). Le immagini provengono sia dal dataset originale del paper (*Architectural Style Classification using Multinomial Latent Logistic Regression*) sia da scraping web (Google Images) per ampliare la varietà [13] . Ogni classe ha qualche centinaio di immagini (ad esempio ~400 gotiche, ~300 romaniche, ~600 Art Nouveau, ecc., con alcune variazioni) [29] [30] . Questo dataset è disponibile su Kaggle (utente *dumitrux*) e su Roboflow Universe (come *ArchiStyles* o *Architecture Styles*). È il riferimento principale per il nostro task, ideale per allenare e valutare modelli di classificazione multi-classe. Dalle esperienze riportate: una CNN ResNet fine-tunata su questo insieme ha raggiunto ~95% di accuracy [15] , mentre un modello ML del 2014 otteneva ~70% [16] , segno dei progressi ottenuti con deep learning e dataset estesi.

- **Kaggle "Architectural Styles" e varianti:** Oltre al dataset di cui sopra, su Kaggle esistono versioni alternative/complementari. Ad esempio, un **"Architectural Styles - Periods"** dataset raccoglie immagini simili (potrebbe essere una riorganizzazione per epoca). Inoltre, il dataset **"International Architectural Styles - Combined"** (utente *jungseolin*) pare combinare più fonti per un totale ancora maggiore di immagini. Queste varianti vanno verificate (possono includere dati ridondanti). Per iniziare, probabilmente conviene utilizzare la versione standard a 25 classi, eventualmente integrandola con altre immagini da queste fonti se necessario per aumentare esempi di determinate categorie.

- **ArchiTective (Roboflow 4-class)** – Un dataset più piccolo, ~**1800 immagini** in 4 categorie (*Art Nouveau, Gothic, Deconstructivism, Palladian*). È stato usato per addestrare un modello disponibile su Roboflow Universe [18] [19] . Può essere utile come **dataset di validazione** o per esperimenti preliminari, dato che è limitato a pochi stili (peraltro inclusi anche nel set a 25 classi). Ad esempio, potremmo provarlo per un test iniziale zero-shot vs fine-tuning su 4 classi, prima di affrontare tutte le 25 del dataset completo.

- **Building Detection (Roboflow)** – Non è un dataset di classificazione di stili, ma di **rilevamento di edifici**. Ad esempio, su Roboflow Universe vi sono piccoli set di immagini con bounding box di edifici (es. **Building Detector** con ~250 immagini annotato per detection) [31] . Lo scopo di questi dati, nel nostro progetto, è di supporto al preprocessing: possiamo usare un modello di detection pre-addestrato (disponibile via API Roboflow oppure modelli open come YOLO) per individuare l'edificio principale nella foto e ritagliarlo. Ciò elimina porzioni di cielo, strade,

persone ecc. che possono fare rumore durante la classificazione. Dunque, pur non contribuendo direttamente all'addestramento dello classificatore di stili, questo dataset "Building Detector" è **complementare** per addestrare o utilizzare un rilevatore di edifici se le nostre foto non sono già ben centrate sul soggetto.

- **Dataset di Elementi Architettonici (AHE Dataset):** Un'altra risorsa interessante proviene dal dominio dei beni culturali. Un gruppo di ricerca ha creato un **Architectural Heritage Elements Dataset** con **~10.235 immagini** di elementi architettonici storici (come finestre, colonne, volte) suddivisi in categorie stilistiche [32]. Lo scopo originale era classificare elementi architettonici per epoca/stile, a supporto di studi sull'evoluzione degli stili. Questo dataset (pubblicato su DataHub [33]) potrebbe essere sfruttato per **data augmentation** concettuale: ad esempio, allenare CLIP a riconoscere dettagli architettonici tipici (un rosone gotico, un capitello corinzio, etc.) che poi aiutino a determinare lo stile di un edificio completo. Potenzialmente, si potrebbe combinarlo come ulteriore fonte di immagini: però bisogna valutare se le classi di elementi corrispondono direttamente agli stili (spesso ogni elemento è etichettato con stile/periodo, quindi potrebbe arricchire la conoscenza del modello su specifici dettagli di quel periodo).

- **Altre fonti:** Si potrebbe considerare l'utilizzo di **Google Images/Bing** per cercare immagini addizionali per stili poco rappresentati. Essendo però un progetto da sviluppare preferibilmente con dataset noti, conviene prima esaurire quelli esistenti. In ambito accademico, alcune pubblicazioni hanno lavorato su **dataset geo-localizzati** – ad esempio foto di architetture di determinate città con l'obiettivo di classificarne lo stile o l'epoca. Uno studio recente ha compilato un dataset di facciate da Google Street View per Amsterdam e Stoccolma, con annotazioni di periodo storico [34], per studiare l'evoluzione temporale degli stili urbani. Sebbene tali dataset siano molto specifici (e non sempre pubblici), indicano che esistono opportunità di dataset *specializzati per area geografica*. Se il nostro obiettivo fosse migliorare il riconoscimento per un certo contesto (es. architetture italiane), potremmo integrare immagini di quel contesto durante il fine-tuning, per tarare il modello su varianti locali degli stili (p. es. il *Gotico italiano* vs *Gotico francese*).

In sintesi, il dataset di base consigliato è quello Kaggle/Roboflow a 25 classi. Eventualmente, si può **ampliare** con immagini aggiuntive da fonti open (rispettando licenze) per classi con pochi esempi. L'importante è mantenere un buon **split di validazione/test** con immagini non viste, per valutare realisticamente le prestazioni.

Pipeline di Elaborazione e Best Practices

Mettiamo insieme i pezzi per delineare la pipeline completa del progetto e alcune best practice per ciascuna fase, dalla pre-elaborazione alla valutazione:

1. Preprocessing delle Immagini

Una buona **preparazione dei dati** migliora sia l'accuratezza del modello che la sua robustezza. Nel contesto delle foto di edifici, suggeriamo di implementare questi passi:

- **Rilevamento e Ritaglio dell'Edificio:** Utilizzando un modello di object detection (ad esempio YOLOv5 addestrato sul dataset *Building Detector* di Roboflow [31]), individuare la bounding box dell'edificio principale nell'immagine. Ritagliare o mascherare lo sfondo intorno all'edificio permette al classificatore di concentrarsi sugli elementi architettonici rilevanti, evitando confusione dovuta a cielo, vegetazione o altri edifici circostanti. Se le foto del dataset sono già

ben centrate sull'edificio, questo passo è facoltativo; ma per immagini "in the wild" è fortemente consigliato. In output di questa fase otterremo immagini normalizzate dove l'edificio occupa la gran parte del frame.

- **Correzione Prospettica:** Le foto possono essere scattate con angolazioni molto oblique (ad esempio dal basso verso l'alto su un grattacielo), deformando la percezione delle forme. Applicare una **trasformazione prospettica** per rettificare l'immagine può aiutare il modello a riconoscere meglio elementi chiave come proporzioni di finestre, archi simmetrici, etc. In OpenCV, ad esempio, si può utilizzare una trasformazione di prospettiva definendo quattro punti di riferimento (ad es. gli angoli dell'edificio se identificabili) e rimappandoli a un rettangolo diritto [35]. Questo allineamento rende la facciata "diritta" nell'immagine risultante, facilitando l'analisi delle caratteristiche architettoniche come se l'osservatore fosse di fronte all'edificio. Naturalmente, non sempre è possibile una correzione perfetta (dipende dall'immagine e dalla possibilità di stimare il piano), ma anche solo compensare parzialmente l'inclinazione (es. rendere verticali le linee di struttura con strumenti come **rotation correction** o proiezione parallela) può giovare. In letteratura, l'allineamento delle facciate è considerato un fattore chiave per confrontare stili, perché elimina variabilità non legata allo stile.

- **Filtri di Miglioramento e Pulizia:** È opportuno applicare tecniche di miglioramento dell'immagine come **riduzione del rumore** (es. filtro bilaterale se ci sono artefatti, soprattutto se alcune immagini provengono da scansioni di foto storiche), e **miglioramento del contrasto** (ad esempio histogram equalization) per evidenziare dettagli. Bisogna applicare questi filtri con moderazione per non alterare i colori originali in modo da confondere il modello (alcuni stili sono caratterizzati anche dai materiali/colori usati). Se il dataset include immagini in bianco e nero (edifici storici d'epoca), valutarne la conversione in scala di grigi per uniformità, o al contrario lasciare colori se il modello può trarne informazione (ad esempio, lo stile *Bauhaus* spesso presenta facciate bianche lisce: il colore potrebbe essere indicativo).

- **Ridimensionamento e Normalizzazione:** Come ultimo step di preprocessing, portare tutte le immagini ad una dimensione uniforme compatibile con CLIP (solitamente 224×224 pixel per CLIP ViT-B/32, o 224×224 / 336×336 per modelli maggiori). Utilizzare il **processor ufficiale di CLIP** (fornito da HuggingFace `AutoProcessor` o le trasformazioni del repository OpenAI) che applica il ridimensionamento, center crop e normalizzazione dei pixel nei canali (CLIP richiede una normalizzazione specifica dei valori RGB). Questo assicura che l'immagine sia nel formato atteso dall'encoder di CLIP.

- **Data Augmentation:** In fase di training, è buona norma applicare augmentation per aumentare la diversità degli esempi e ridurre l'overfitting. Oltre alle trasformazioni geometriche classiche (flip orizzontale, leggera rotazione, piccole traslazioni e zoom), nel nostro caso può essere utile simulare **variazioni di illuminazione** (dato che edifici possono essere fotografati all'alba, pieno giorno, sera), differenti condizioni meteo (un cielo bianco vs blu potrebbe influire sui colori percepiti dell'edificio) e **occlusioni parziali** (ad es. crop parziali, come se parte dell'edificio fosse nascosta da alberi o altro – questo insegna al modello a non dipendere dall'intera facciata visibile). L'augment può essere implementato con librerie come Albumentations o direttamente con trasformazioni PyTorch. Lo scopo è generare immagini sintetiche variate: è stato osservato che su ArchiStyles l'augment aumenta sensibilemnte l'accuratezza finale [14]. **NB:** Evitare augmentation che stravolgano lo stile: ad esempio filtri di **inversione colori** o eccessive distorsioni geometriche non realistiche, perché potrebbero alterare i pattern architettonici (ad esempio, allungare troppo un elemento potrebbe farlo somigliare ad un altro stile). In generale, mantenere trasformazioni plausibili.

2. Estrazione di Embedding con CLIP

Con le immagini preprocessate, il prossimo stadio è ottenere gli **embedding CLIP**. Best practice qui includono:

- **Utilizzo del modello appropriato:** Se si è optato per CLIP come backbone, scegliere la variante più adatta. CLIP ViT-B/32 è veloce e relativamente leggero, ma CLIP ViT-L/14 offre maggiore accuratezza (avendo un modello più capace e immagini input 336px). Si potrebbe iniziare col B/32 per prototipazione e poi valutare L/14 se serve boost di performance e se si hanno risorse. In PyTorch/HuggingFace, caricare il modello con `CLIPModel.from_pretrained("openai/clip-vit-base-patch32")` (o il nome corrispondente). Impostare il model in eval mode (disabilitando gradienti se all'inizio si fa solo inferenza zero-shot o feature extraction).

- **Batching e GPU:** Estrarre embedding per un intero dataset può essere dispendioso; utilizzare batch di dimensioni adeguate alla GPU disponibile (ad es. batch size 32 o 64, a seconda della VRAM). HuggingFace AutoProcessor può preparare direttamente un batch di immagini; in alternativa, se abbiamo le immagini come Tensor PyTorch già trasformate, passarle direttamente al `model.get_image_features(...)` se disponibile, o al forward del modello. È essenziale inoltre **normalizzare gli embedding risultanti** (CLIP tipicamente normalizza i vettori a norma unitaria durante training, ma in inferenza conviene farlo esplicitamente) prima di usarli per confronto coseno.

- **Modalità zero-shot (opzionale):** Se stiamo eseguendo classificazione zero-shot, per ogni immagine calcoleremo i logits rispetto alle class queries. In pratica:

- Preparare le frasi prompt per ogni stile, es. *"a photo of a Gothic building"*, *"a photo of a Baroque building"*, etc. (La scelta della lingua può influire: CLIP originale è addestrato principalmente su testi in inglese, quindi è preferibile usare prompt in inglese per massimizzare la familiarità del modello con i termini degli stili. Termini come *"Gothic architecture"* dovrebbero essere compresi; se qualche stile ha nomi in altre lingue (es. *"Bauhaus"* è già parola tedesca ma diffusa in inglese) valutare la versione più nota in inglese).
- Passare la lista di testi e l'immagine al modello come mostrato prima [36], ottenendo `logits_per_image`.
- Identificare l'indice del logit massimo e mappare al nome dello stile corrispondente. Quello sarà la predizione. Si può anche ottenere una distribuzione di probabilità applicando softmax [5], utile per valutare la *confidence* del modello o avere un ranking di top-N stili più probabili.

- Se multi-label fosse rilevante, si potrebbe decidere una soglia e assegnare tutte le classi la cui probabilità supera tale soglia. In alternativa, interrogare il modello con domande diverse, ma questo complicherebbe l'uso zero-shot. Di base, CLIP zero-shot dà un output tipo *softmax* su quelle N classi fornite.

- **Uso come feature extractor (per fine-tuning):** Se invece procediamo con fine-tuning tradizionale, gli embedding estratti da CLIP saranno passati ad un modello classificatore. Con HuggingFace, `CLIPModel` restituisce sia logits immagine-testo sia i tensori interni; ma possiamo semplicemente prendere l'**ultimo strato dell'encoder immagini**. Ad esempio, con `CLIPModel`, `outputs.image_embeds` fornisce l'embedding denso (di dimensione 512 per ViT-B/32). Alternativamente, si può usare `CLIPVisionModel` per ottenere direttamente il visual features vector. Durante il training, se fine-tuniamo, combineremo CLIP (o almeno la sua parte vision) con uno strato finale di dimensione [embedding_dim -> numero_classi] che produce le logit per le nostre classi. In questa fase, è fondamentale monitorare l'**andamento del training**

(loss e accuracy su validation) per evitare overfitting: CLIP potrebbe arrivare ad overfit dopo poche epoche, quindi conviene usare early stopping o tenere un occhio sul gap train-val.

- **Strategie di fine-tuning:** Come accennato, una strategia comune è il *two-phase training*: iniziare allenando solo la **testa classificatrice** (con CLIP congelato), per alcune epoche finché il loss cala; poi sbloccare alcuni degli ultimi layer dell'encoder CLIP e continuare il training a learning rate molto basso. Questo consente di *adattare gradualmente* il modello senza stravolgerlo subito. Si può anche usare tecniche come **gradual unfreezing** (sbloccare progressivamente più layer) e **discriminative learning rates** (LR più alto per i layer finali, più basso per quelli iniziali – la libreria FastAI ad esempio agevola questo). Dato che CLIP è un transformer, si può anche considerare il fine-tuning con approcci più leggeri come **LoRA** (Low-Rank Adapters) o **Prompt Tuning** per viz parte, ma sono avanzati e forse eccessivi per questo caso: un classico fine-tune end-to-end andrà bene se fatto con cura.

3. Classificazione Finale e Integrazione del Retrieval

Dopo aver estratto (e eventualmente fine-tunato) le feature CLIP, si procede alla **classificazione** vera e propria:

- **Classificazione multi-classe:** Nel caso standard, il modello (sia esso CLIP+MLP fine-tunato o CLIP zero-shot con prompt) fornirà una singola etichetta di stile per ogni immagine. Si può arricchire l'output restituendo anche un punteggio di confidenza. Ad esempio, se usiamo CLIP zero-shot e troviamo due stili con punteggi vicini, potremmo indicare l'incertezza. Nel contesto applicativo, ciò potrebbe essere usato per eventualmente fare un secondo passaggio: ad esempio, se il modello è incerto tra due stili, una logica aggiuntiva potrebbe attivare una ricerca per immagini simili (retrieval) o richiedere conferma da un esperto. Ma questo va oltre il puro riconoscimento automatico.

- **Classificazione multi-label (se applicabile):** Se decidiamo di permettere più stili per immagine (scenario non comune, ma possibile ad esempio per edifici con ristrutturazioni di epoche diverse), allora l'output diventa un **set di etichette**. In fase di training, ciò comporta usare una loss come *binary crossentropy per etichetta* invece di softmax crossentropy, e soglie per predire più classi. In pratica, per ogni stile il modello prevede un punteggio indipendente (sigmoide invece di softmax) e si decide un threshold per dire "stile presente". Questa configurazione richiede un dataset dove magari siano annotate più etichette per immagine (il che non so se è il caso di ArchiStyles – probabilmente no, ogni edificio ha una sola etichetta principale). Se non abbiamo tali annotazioni, probabilmente resteremo con l'assunzione multi-classe. In ogni caso, CLIP può essere usato anche multi-label: ad esempio generando embedding per ogni singola classe e facendo detection di *tutti* quelli con similarità sopra un certo valore. Ma teniamo presente che la maggior parte dei dataset e letteratura su architectural style assume classificazione singola.

- **Integrazione del modulo di retrieval:** Se abbiamo implementato la fase di image retrieval, possiamo combinarla con la classificazione in diversi modi. Un semplice è il **Nearest Neighbor Voting**: invece di prendere l'output del classificatore, cerchiamo i K esempi più simili (embedding) nel database di riferimento; facciamo una votazione a maggioranza o ponderata sulle loro etichette di stile. Questo può migliorare robustezza se il database è ampio e affidabile. Oppure possiamo usare il risultato del retrieval come spiegazione: "*Predicted Gothic (confidence 0.85). Top-3 similar images from database are also Gothic, corroborating la previsione*". Dal punto di vista di pipeline, questo step richiede di avere pre-calcolato e memorizzato gli embedding CLIP di tutte le immagini del database di stili. Tali embedding andrebbero aggiornati se mai il modello

CLIP venisse fine-tunato (perché cambierebbe lo spazio delle feature). Se CLIP è usato frozen, possiamo calcolarli una volta e riutilizzarli. Per la ricerca veloce si può utilizzare un indice approssimato (faiss) se i dati sono tanti.

4. Valutazione e Metriche

La valutazione deve essere rigorosa, utilizzando un insieme di **test** separato (idealmente mai visto durante lo sviluppo, per avere stime realistiche). Le metriche da considerare:

- **Accuracy:** Percentuale di immagini correttamente classificate. È la metrica più immediata per un problema multi-classe bilanciato, ma occorre attenzione se il dataset ha classi sbilanciate (alcuni stili molto più rappresentati di altri). Nel caso di ArchiStyles, le classi hanno dimensioni abbastanza comparabili (tutte qualche centinaio di esempi, con la più grande ~720 immagini e la più piccola ~278) [37] [38], quindi l'accuracy è significativa. In progetti precedenti, come citato, sono state raggiunte accuracy intorno al 90% o più su questo task [15].

- **Precision, Recall, F1-score:** Per una valutazione più granulare, calcolare precisione e richiamo per ciascuno stile. Questo aiuta a capire se il modello confonde certi stili specifici. Ad esempio, se il richiamo per "Romanico" è basso significa che molti edifici romanici vengono classificati come qualcos'altro (forse Gotico?), suggerendo confusione tra stili consecutivi. **F1** (media armonica di precision e recall) può essere riportato come singolo numero per classe e poi **macro-averaged** su tutte le classi per dare una misura che pesi allo stesso modo ogni stile. Se alcune classi fossero molto minoritarie, la *macro-F1* penalizzerebbe le prestazioni scarse su di esse più di quanto faccia l'accuracy grezza.

- **Matrice di Confusione:** È buona pratica ispezionare la confusion matrix delle predizioni, per identificare pattern di errore. Ad esempio, potremmo scoprire che quasi tutti gli edifici *Queen Anne* vengono scambiati per *Victorian* (ipotetico), il che avrebbe senso essendo stili correlati. Questo insight potrebbe portare a rifinire i prompt (se CLIP zero-shot, forse includere "Queen Anne victorian style" come frase) o aggiungere più immagini di training per quelle due classi per aiutare la distinzione. La matrice di confusione visivamente può guidare miglioramenti specifici della pipeline.

- **Top-K Accuracy / Recall@K:** Dato che a volte gli stili possono essere ambigui, si può misurare la percentuale di casi in cui l'etichetta corretta è tra le **prime K** proposte dal modello. Ad esempio, un modello potrebbe non avere "Romanico" come prima predizione ma come seconda (dopo "Gotico"), magari perché l'immagine era ambigua. Avere una buona top-2 o top-3 accuracy significa che il modello raramente "perde completamente" lo stile corretto – scenario utile se si vuole fornire all'utente una lista di possibili stili. In un esperimento sui dati di heritage, è stato riportato un **recall del 96.76% entro le top-2 classi** previste [39], il che indica che spesso l'etichetta giusta era la seconda scelta del modello. Misurare Top-2, Top-3 accuracy nel nostro caso può essere informativo allo stesso modo. Per la modalità multi-label, equivalenti sarebbero metriche tipo **Precision@K** o **Mean Average Precision (mAP)** su tutte le etichette.

- **Metriche specifiche per retrieval (se implementato):** Se includiamo la fase di retrieval, possiamo valutare anche quella con metriche di information retrieval: ad esempio, per ogni immagine query, la proporzione di immagini dello stesso stile nei suoi top-5 vicini (Precision@5), o il recall se il dataset di riferimento ha più esempi dello stesso stile. Possiamo anche confrontare un approccio K-NN puro vs il classificatore per vedere quale performa meglio o se combinati danno vantaggi.

- **Robustezza e test qualitativi:** Oltre ai numeri, è utile fare qualche test qualitativo: visualizzare alcune predizioni difficili e capire perché magari il modello ha sbagliato. Questo potrebbe rivelare, ad esempio, che certe foto sono di dettagli e non dell'intero edificio (magari il modello confonde un primo piano di una finestra gotica con stile gotico anche se l'edificio intero era neogotico – però in quel caso è giusto!). Oppure scoprire che certe etichette del dataset sono rumorose/incoerenti, e il modello in realtà ha ragione a "sbagliare" secondo l'etichetta se quest'ultima era errata. In sintesi, mantenere un occhio critico sui risultati e non solo ottimizzare le metriche.

5. Considerazioni Finali e Risorse Utili

In fase di implementazione, tenere presente anche: - **Risorse Computazionali:** Se si allena CLIP, sfruttare GPU con mixed precision (`fp16`) per velocizzare e ridurre memoria (HuggingFace trainer lo facilita). Per dataset modesti come 10k immagini, il training non dovrebbe essere proibitivo (poche epoche su ViT-B/32 potrebbero convergere in <1 ora su GPU moderna). - **Salvataggio di Modello ed Embedding:** Dopo l'addestramento, salvare il modello fine-tunato. Inoltre, se si è creato un database di embedding (per retrieval o semplicemente per analisi), salvare anche quelli con riferimenti alle rispettive immagini/etichette. - **Documentazione e Link Utili:** Per ulteriori dettagli sull'uso di CLIP in PyTorch, la documentazione HuggingFace CLIP [40] [36] e il blog OpenAI sono risorse chiave. Repositories come `open_clip` [41] forniscono implementazioni ottimizzate e modelli addizionali (OpenCLIP ha anche modelli addestrati su dati LAION più grandi, che potrebbero in futuro essere sfruttati se si vuole provare una variante di CLIP ancora più potente). - **Community insights:** Post su forum (es. discussioni su HuggingFace [42] o Medium blog [6] [7]) spesso affrontano problemi comuni e "tricks" durante il fine-tuning di CLIP, come scelta del learning rate ideale (spesso 1e-5 ~ 5e-6 per il backbone), utilizzo di scheduler (cosine decay), e importanza del prompt ensembling (provare più versioni di prompt e fare media dei logit).

Con queste linee guida, siamo in grado di costruire una pipeline robusta per il riconoscimento dello stile architettonico. In sintesi, **preparazione accurata dei dati**, **sfruttamento intelligente di CLIP** (sia esso zero-shot o adattato via fine-tuning) e **valutazione approfondita** con le giuste metriche costituiranno le chiavi del successo del progetto. Buon lavoro!

Riferimenti e Risorse:

- CLIP: paper OpenAI (2021) e repository ufficiale; Documentazione HuggingFace Transformers [4] [5].
- Tutorial Zero-Shot Image Classification con CLIP [6] [7].
- ArchiStyles dataset e progetto FastAI su Kaggle/GitHub [13] [14].
- Blog *Fine-Tuning CLIP for Image Classification* con confronto ResNet50 [9] [43].
- Roboflow Universe – ArchiStyles e ArchiTective [17] [18] (dataset e modelli pre-addestrati).
- Architectural Heritage Elements Dataset (AHE) – pubblicato in [Llamas et al. 2017], disponibile su datahub [32].
- GitHub MuhammadAliS/CLIP – Esempi di fine-tuning CLIP in PyTorch [22].
- Approfondimenti su progetti di classificazione architettura: Medium di Arshita Bhatt (2020) sul trasfer learning per stili [12], paper ArchiCLIP (CAADRIA 2024) [23].

---

[1] [2] [3] [8] [9] [26] [27] [28] [43] Fine-Tuning the CLIP Foundation Model for Image Classification - Alexander Thamm
https://www.alexanderthamm.com/en/blog/fine-tuning-the-clip-foundation-model-for-image-classification/

4 5 36 40 CLIP
https://huggingface.co/docs/transformers/en/model_doc/clip

6 7 20 Zero-shot image classification with CLIP and Huggingface | by Mert | Medium
https://medium.com/@Mert.A/zero-shot-image-classification-with-clip-and-huggingface-c73c438fce6d

10 22 GitHub - MuhammadAliS/CLIP: PyTorch implementation of OpenAI's CLIP model for image classification, visual search, and visual question answering (VQA).
https://github.com/MuhammadAliS/CLIP

11 13 14 15 16 29 30 37 38 GitHub - dumitrux/architectural-style-recognition: ArchitecturalStyle Recognition is a image classifier of 25 different architectural styles, using deep learning.
https://github.com/dumitrux/architectural-style-recognition

12 What is the architecture style of this building? | by Arshita Bhatt | Medium
https://medium.com/@aarshibhatt112/what-is-the-architecture-style-of-this-building-6170566ad1c1

17 ArchiStyles Classification Dataset and Pre-Trained Model by mipt4
https://universe.roboflow.com/mipt4/archistyles

18 ArchiTective: Architectural Style Image Classification Model ...
https://universe.roboflow.com/arch-styles/architective-architectural-style-image-classification-model-db0jd

19 Version - Roboflow Universe
https://universe.roboflow.com/test-1-architective-mvp/architective-architectural-style-image-classification-model/dataset/6

21 A Guide to Fine-Tuning CLIP Models
https://www.marqo.ai/course/fine-tuning-clip-models

23 24 25 197
https://caadria2024.org/wp-content/uploads/2024/04/197-ARCHICLIP.pdf

31 Building Detector Object Detection Dataset and Pre-Trained Model ...
https://universe.roboflow.com/gemaimodels/building-detector-sxirj

32 33 39 Classification of Architectural Heritage Images Using Deep Learning Techniques
https://www.mdpi.com/2076-3417/7/10/992

34 Understanding Building Age and Style through Deep Learning | Maoran Sun
https://www.maoran.io/projects/buildingAge.html

35 OpenCv Perspective Transformation | by Raqueeb Shaikh - Medium
https://medium.com/analytics-vidhya/opencv-perspective-transformation-9edffefb2143

41 mlfoundations/open_clip: An open source implementation of CLIP.
https://github.com/mlfoundations/open_clip

42 Fine tuning CLIP Transformer for downstream task
https://discuss.huggingface.co/t/fine-tuning-clip-transformer-for-downstream-task/42867